مسار

برنامج تدريب ميداني

2025

Introduction to Linux Operating System Part 2

2

Agenda

- Linux Sudo privileges
- Linux Package Management
- Linux Services
- Linux Process
- Linux Cron Jobs
- Local DNS
- Linux Shell Types
- Bash Scripting

Linux Sudo Privileges

Definition of sudo:

Sudo (superuser do), grants users the ability to execute commands with elevated privileges.

Purpose of sudo:

Balances security and convenience by enabling users to perform administrative tasks without direct root access.

/etc/sudoers:

Defines the sudo (superuser do) command's behavior, specifying which users or groups are allowed to run commands with elevated privileges.

Linux Sudo Privileges - Format

Sudoers file format:

- <username> <host>=(runas) <tags>: <command>
  - **username:** The user or a group of users.
  - **host:** The host or hosts on which the rule is effective. It can be specified as ALL to apply to all hosts.
  - **(runas):** The (<user>:<group>) as whom the command should be run. It can be specified as ALL to allow running as any user.
  - **tags:** Options that modify the sudo behaviour.
  - **command**: The command or commands that the user can execute with sudo.

Linux Sudo Privileges - Examples

Sudo Privilege Examples:

- skidz ALL=(ALL:ALL) NOPASSWD: /usr/bin/cat
  - This line allows the user "skidz" to run /usr/bin/cat without entering a password (NOPASSWD) as any user.
- skidz ALL=(ALL:ALL) NOPASSWD: /usr/bin/cat /etc/shadow
  - means that the user "skidz" is allowed to run the cat command specifically on the file /etc/shadow without entering a password as any user.
- %admins ALL=(ALL:ALL) ALL
  - This line allows members of the "admins" group to execute any command as any user. The % indicates a group.
- skidz ALL=(bob) /bin/cat
  - This line allows the user "skidz" to run /bin/cat as the user "bob."

Linux Sudo Privileges - Commands

Sudo Command: allows a permitted user to execute a command as the superuser (root) or another user, as specified by the security policy configured in the /etc/sudoers file.

Listing the user's sudo privileges:

- `sudo -l`

Using sudo privileges:

- skidz ALL=(ALL:ALL) /bin/cat
  - `sudo /bin/cat`
- skidz ALL=(bob) /bin/cat
  - `sudo -u bob /bin/cat`

Linux Package Management

Definition: Linux package management is a system for installing, updating, configuring, and removing software packages on a Linux system.

Purpose: Simplifies software installation, ensures dependencies are met, and facilitates efficient system maintenance.

Package Formats: Common package formats include DEB (used by Debian-based systems like Ubuntu and kali) and RPM (used by Red Hat-based systems like Fedora).

Package Managers: Popular package managers include APT (Advanced Package Tool) for Debian-based systems and YUM (Yellowdog Updater Modified) for Red Hat-based systems.

Linux Package Management

Installation:

- Debian-based systems: `sudo apt-get install <package_name>`
- Red Hat-based systems: `sudo yum install <package_name>`

Update Packages:

- Debian-based systems: `sudo apt-get update`
- Red Hat-based systems: `sudo yum update`

Removal:

- Debian-based systems: `sudo apt-get remove <package_name>`
- Red Hat-based systems: `sudo yum remove <package_name>`

Linux Services

Definition: Linux services are background processes or daemons that run independently of user interactions, providing essential system functions or network services.

Purpose: Services handle tasks such as networking, file sharing, printing, and more, ensuring continuous operation and responsiveness.

Service States: Services can be in various states, including running, stopped, or restarting.

Linux Services File Structure

```
[Unit]
Directive= <The unit's meta data and its relationship with other units>

[Service]
Directive= <The service and commands used to start the service>

[Install]
Directive= <(Optional) Configure the behaviour or services that are enabled or disabled>
```

These structures and directives are well defined and case-sensitive, so be cautious!

Linux Services - /etc/services

The /etc/services file in Linux is a plain text file that maps network service names to their corresponding port numbers and protocols (TCP or UDP).

```
http    80/tcp   www www-http   # World Wide Web HTTP
ftp     21/tcp                  # File Transfer Protocol
ssh     22/tcp                  # SSH Remote Login Protocol
dns     53/tcp   domain         # Domain Name Server
dns     53/udp   domain
```

Linux Services - Commands

Service Commands:

- Start a service: `sudo systemctl start <service_name>`
- Stop a service: `sudo systemctl stop <service_name>`
- Restart a service: `sudo systemctl restart <service_name>`
- Reload a service: `sudo systemctl reload <service_name>`

Enable/Disable at Boot:

- Enable a service at boot: `sudo systemctl enable <service_name>`
- Disable a service at boot: `sudo systemctl disable <service_name>`

Linux Services - Commands

Check Service Status:

- View service status: `sudo systemctl status <service_name>`

Log Files:

- Check service logs: `journalctl -u <service_name>`

Check Running Services:

- View running services: `systemctl --type=service --state=running`

Linux Processes

Definition:

In Linux, a process is an instance of a running program. It consists of the program code, data, and system resources allocated during its execution.

Process Identification:

- Each process is assigned a unique Process ID (PID) for identification.
- Parent and child processes are created through the fork() system call.

Process States:

Processes can be in various states: Running, Sleeping, Stopped, or Zombie.

Linux Processes - Commands and Management

Process Commands:

- View running processes: `ps aux` or `top`
- Kill a process by PID: `kill <PID>` or `kill -9 <PID>` (forceful termination)

Background (daemon) and Foreground Processes:

- Run a process in the background: `<command> &`
- View background processes with their ID: `jobs`
- Bring a background process to the foreground: `fg %<ID>`

Linux Processes - Prioritization and Monitoring

Process Prioritization:

- Adjust process priority: `nice` or `renice`

Process Monitoring:

- Monitor real-time process activity: `htop`

Linux Processes - Running Processes Example

```
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
kali        1461  2.1  0.2 266028 26128 ?        Sl   16:14   0:00 xfce4-power-manager --restart --sm-client-id 27de0bede-6d01-451d-8f7d-7a
kali        1464  2.7  0.2 853320 25576 ?        SNsl 16:14   0:00 /usr/lib/x86_64-linux-gnu/tumbler-1/tumblerd
kali        1489  0.9  0.1 309532 12088 ?        Sl   16:14   0:00 xiccd
kali        1503  2.4  0.4 635176 50888 ?        Sl   16:14   0:00 nm-applet
kali        1505  0.3  0.1 425448 13728 ?        Ssl  16:14   0:00 /usr/libexec/gvfs-udisks2-volume-monitor
kali        1514  0.1  0.0  14652  4016 ?        Ssl  16:14   0:00 xcape -e Super_L Control_L Escape
kali        1519  1.7  0.2 339068 25380 ?        Sl   16:14   0:00 light-locker
root        1520  0.1  0.0  96808  3540 ?        Ssl  16:14   0:00 /usr/sbin/pcscd --foreground --auto-exit
colord      1523  2.2  0.1 316304 17196 ?        Ssl  16:14   0:00 /usr/libexec/colord
kali        1526  0.0  0.0 230120  4652 ?        Ssl  16:14   0:00 /usr/libexec/dconf-service
root        1539  1.2  0.1 469096 16480 ?        Ssl  16:14   0:00 /usr/libexec/udisks2/udisksd
kali        1540  0.6  0.1 259764 17744 ?        Sl   16:14   0:00 /usr/lib/policykit-1-gnome/polkit-gnome-authentication-agent-1
kali        1557  0.1  0.0 307716  6876 ?        Sl   16:14   0:00 /usr/libexec/geoclue-2.0/demos/agent
kali        1562  7.7  0.4 523644 52828 ?        Sl   16:14   0:00 /usr/bin/python3 /usr/bin/blueman-applet
kali        1563  4.2  0.3 363020 41364 ?        Sl   16:14   0:00 /usr/bin/vmtoolsd -n vmusr --blockFd 3
kali        1577  1.8  0.0  10164  6248 pts/0    Ss+  16:14   0:00 /usr/bin/zsh
kali        1670  0.1  0.0 307132  6852 ?        Ssl  16:14   0:00 /usr/libexec/gvfs-mtp-volume-monitor
kali        1680  0.5  0.0 386960 10296 ?        Ssl  16:14   0:00 /usr/libexec/gvfs-afc-volume-monitor
kali        1700  0.5  0.0 308084  9516 ?        Ssl  16:14   0:00 /usr/libexec/gvfs-gphoto2-volume-monitor
kali        1713  0.1  0.0 307192  6964 ?        Ssl  16:14   0:00 /usr/libexec/gvfs-goa-volume-monitor
kali        1741  0.3  0.0 385720  9576 ?        Sl   16:14   0:00 /usr/libexec/gvfsd-trash --spawner :1.20 /org/gtk/gvfs/exec_spaw/0
kali        1755  0.1  0.0 233604  6948 ?        Ssl  16:14   0:00 /usr/libexec/gvfsd-metadata
root        1784  3.9  0.3 355868 35968 ?        Ssl  16:14   0:00 /usr/bin/python3 /usr/libexec/blueman-mechanism
kali        1788  0.5  0.1  39056 16292 ?        S    16:14   0:00 /usr/bin/xfce4-mime-helper --launch TerminalEmulator
kali        1791  5.2  0.8 456876 100088 ?       Sl   16:14   0:00 /usr/bin/qterminal
kali        1799  2.1  0.0  10356  6588 pts/1    Ss   16:14   0:00 /usr/bin/zsh
kali        1800  0.2  0.0  48924  7308 ?        Ss   16:14   0:00 /usr/libexec/bluetooth/obexd
```

Linux Processes vs Services

Processes:

- A process is any running program or task on the system.
- Can be started manually by a user or automatically by the system.
- Has its own Process ID (PID) and resources (memory, CPU).
- **Examples:** firefox , top , a running script in the terminal.

Services:

- A service is a type of process that runs in the background (a daemon).
- Managed by the system's init system (e.g., systemd).
- Often starts automatically at boot and provides essential system or network functions.
- **Examples:** sshd , nginx , cron .

Linux Processes vs Services

Feature	Process	Service
Purpose	Any running task/program	Long-running background task providing a system function
Control	Manual or system	Usually via systemctl or service commands
Lifetime	Ends when the program exits	Often persistent, runs until stopped or system shutdown
Management	Viewed/Killed with ps, top, kill	Managed via service tools, also visible in ps

Linux Cron Jobs

Cron jobs are scheduled tasks in Unix-like operating systems, including Linux, that are executed automatically at specified intervals.

Common uses include:

1. Run recurring tasks automatically

- **Example:** daily backups, log rotation, or sending reports.

2. Perform system maintenance

- **Example:** clearing temporary files, updating software, or monitoring disk usage.

3. Schedule tasks at precise times

- Can run commands **minutely, hourly, daily, weekly, monthly**, or at any custom schedule.

Linux Cron Jobs

Cron jobs are defined in **crontab files**, which contain the schedule and the command to be executed.

- **User-specific Crontabs**
  - The crontab files for individual users are stored in the /var/spool/cron/crontabs/ directory.
  - Format :
    - m h dom mon dow command
- **System-wide Crontab**
  - The system-wide crontab, which contains entries for system-related tasks, is often located in /etc/crontab
  - Format :
    - m h dom mon dow user command

Linux Cron Jobs - Examples

Cron Jobs Examples:

- User specific
  - 30 15 * * * <command or path to script file>
    - The fields 30 15 * * * represent the minute (30) and hour (15) when the job should run. The asterisks (*) in the day of the month, month, and day of the week fields mean "every day."
- System-wide
  - 17 * * * * root <command or path to script file>
    - In this example, the cron job is scheduled to run every 17th minute of every hour and execute the specified command or script file as root user.

Linux Cron Jobs - Commands

Commands:

- `crontab -e`
  - Edit the current user's crontab
- `crontab -l`
  - List the current user's crontab entries
- `crontab -r`
  - Remove the current user's crontab
- `crontab -u <user> -e/l/r`
  - Same operations above but for a specific user. Requires elevated privileges.

Local DNS - /etc/hosts File

Definition: The hosts file, /etc/hosts, is a local mapping of IP addresses to domain names. It is used to resolve domain names to IP addresses before querying a DNS server.

Syntax:
<IP Address> <Hostname> [Aliases]

Example:
127.0.0.1 localhost
192.168.1.100 myserver.local myserver
8.8.8.8 google.com

Linux Shell Types

Feature	Bash	Zsh	Sh
Scripting Power	Powerful scripting language	Capable of basic scripting	Baseline shell with basic functionality
Extensibility	Extensive features for customization	Themes, plugins, and modular architecture	Lightweight and chosen for efficiency
Interactive Features	Advanced command-line editing	Advanced auto-completion and suggestions	Basic functionality for command execution
Compatibility	Backward compatible with Bourne Shell	Main shell in MacOS	Portable syntax across Unix-like systems

Bash Scripting

Definition: Bash scripting is the process of writing and executing scripts using the Bash shell, a command-line interpreter for Unix-like operating systems.

Purpose:

- Automate Repetitive Tasks
- System Administration
- Customization

Scripting Elements:

- Variables
- Control Structures
- Functions

Bash Scripting

Shebang Line: The first line of a script, beginning with '`#!/bin/bash`', indicating the script should be executed using the Bash interpreter.

Variable Assignments:

```
variable_name="value"
```

Conditional Statements:

```
if [ condition ]; then
  # code block
elif [ another_condition ]; then
  # code block
else
  # code block
fi
```

Bash Scripting

Loops:

- **For Loop:**

```
for item in list; do
    # code block
done
```

- **While Loop:**

```
while [ condition ]; do
    # code block
Done
```

Functions:

```
function_name() {
  # code block
}
```

Calling the Function:

```
function_name
```

Bash Scripting - Key Concepts

1. Command Substitution:

Capture command output into a variable.

- **Example:**

```
current_date=$(date)

echo "The system date and time is: $current_date"
```

2. The $PATH Variable

Lists directories where shell looks for commands

- **Example:**

```
echo $PATH
```

- **Output Example**

```
/usr/local/bin:/usr/bin:/bin
```

Bash Scripting - Key Concepts

3. Environment Variables

- System-wide variables available to all programs
- used to store system information and configuration.

- **Common Environment Variables:**
  - $USER: Current logged-in user.
  - $HOME: Home directory of the current user.
  - $PWD: Present Working Directory.
  - $SHELL: Path to the current shell.

- **Using them in a script:**

```
#!/bin/bash

echo "Welcome, $USER! Your home directory is $HOME."

echo "You are currently in $PWD and using the $SHELL shell."
```